

# AHMI Dynamic Route ETA Billboard System

## Technical Setup Guide — Final Modular Asset

| Item              | Value                                                                           |
|-------------------|---------------------------------------------------------------------------------|
| Target engine     | Unity project with Transform-based route nodes                                  |
| Main purpose      | Display stable, dynamic ETA and pickup information on ceiling billboards        |
| Integration model | External orchestrators provide route, pickup label, and traffic values          |
| Asset scope       | ETA calculation, billboard update, route bridge, traffic buffer, route metadata |

## 1. System Purpose

This asset implements a reusable ETA and billboard communication system for an autonomous parking scenario. It receives a dynamic route from an external parking/pickup orchestrator, estimates the vehicle arrival time, applies route and traffic delays, and updates one or more ceiling-mounted billboard interfaces.

- No dependency on fixed WP\_\* folders or hardcoded route names.
- No dependency on a fixed parking spot or fixed pickup area.
- Designed to be exported as a Unity Package and rewired in different project versions.

## 2. Runtime Architecture

External Parking / Pickup Orchestrator

- > `DynamicRouteBridge.ReceiveRoute(routeNodes, pickupLabel)`
- > `CarETAOrchestrator.SetDynamicRoute(...)`
- >  $\text{ETA} = \text{base travel time} + \text{route delays} + \text{traffic buffer} + \text{safety buffer}$
- > `SimulationEvents.RaiseETAUpdated(...)`
- > `BillboardReceiver` updates `TextMeshPro` UI

### 2.1 Data responsibilities

| Component                          | Responsibility                                                                 |
|------------------------------------|--------------------------------------------------------------------------------|
| External random route orchestrator | Chooses parking spot, pickup area, and ordered route nodes.                    |
| Main traffic orchestrator          | Provides <code>totalCarsInSimulation</code> and <code>activeCarsCount</code> . |
| <code>DynamicRouteBridge</code>    | Converts external route outputs into the generic ETA system input.             |
| <code>CarETAOrchestrator</code>    | Calculates ETA from the current route and delay providers.                     |
| <code>BillboardReceiver</code>     | Displays ETA, pickup label, and vehicle status.                                |

## 3. Folder Structure

```
Assets/  
  Scripts/  
    Events/  
      SimulationEvents.cs  
    Car/  
      CarETAOrchestrator.cs  
  Billboard/  
    BillboardReceiver.cs  
  Interaction/  
    DynamicRouteBridge.cs  
    DynamicRouteDebugTester.cs  // optional debug only  
    RouteEndpoint.cs  
    RouteNodeMetadata.cs  
    ParkingTrafficProvider.cs  
    TotemCarRequest.cs          // optional interaction trigger
```

## 4. Recommended Scene Hierarchy

```
AHMI_DynamicRouteSystem
  DynamicRouteBridge
  ParkingTrafficProvider
  DynamicRouteDebugTester  // optional
```

```
Car_Autonomous
  Rigidbody
  CarETAOrchestrator
  Car_TriggerCollider
  Box Collider
```

```
Billboard_A
  Billboard_Screen_Canvas
  Background_Image
  ETA_Text
  PickupArea_Text
  Status_Text
  BillboardReceiver
```

```
ParkingSpot_X
  RouteEndpoint
```

```
Pickup_A / Pickup_B / Pickup_C
  RouteEndpoint
```

```
PedestrianCrossing_Node
  RouteNodeMetadata
```

## 5. Setup Steps

### 5.1 Main system object

1. Create an empty GameObject named AHMI\_DynamicRouteSystem.
2. Add DynamicRouteBridge.
3. Add ParkingTrafficProvider.
4. Optionally add DynamicRouteDebugTester for isolated tests before connecting the real orchestrator.

### 5.2 Autonomous car

5. Select the root vehicle object, for example Car\_Autonomous.
6. Add Rigidbody and set Is Kinematic = true, Use Gravity = false, Interpolate = Interpolate, Collision Detection = Continuous Speculative.
7. Add CarETAOrchestrator to the same root object.
8. Create a child object named Car\_TriggerCollider.
9. Add Box Collider to Car\_TriggerCollider and enable Is Trigger.

### 5.3 CarETAOrchestrator configuration

| Field                                     | Recommended value       | Purpose                                                   |
|-------------------------------------------|-------------------------|-----------------------------------------------------------|
| Speed Meters Per Second                   | 3                       | Average vehicle speed used for base travel time.          |
| Waypoint Reach Threshold                  | 1                       | Distance threshold used to consider a route node reached. |
| Parking Traffic Provider                  | AHMI_DynamicRouteSystem | Drag the object containing ParkingTrafficProvider.        |
| Default Pedestrian Crossing Delay Seconds | 8                       | Delay added for crossing nodes without override.          |
| Safety Buffer Seconds                     | 5                       | Fixed extra time to avoid overly optimistic ETA.          |
| Internal Calculation Interval             | 1                       | Internal ETA refresh interval.                            |
| Billboard Publish Interval                | 10                      | Minimum interval between visible billboard updates.       |

|                              |       |                                                       |
|------------------------------|-------|-------------------------------------------------------|
| ETA Change Threshold Seconds | 15    | Publish early if ETA changes by at least this amount. |
| Round ETA To Nearest Seconds | 30    | Rounds displayed ETA upward for stable UX.            |
| Log Debug Info               | false | Enable only for debugging.                            |

5.4 Billboard UI

- 10. Create a child Canvas under the billboard object.
- 11. Set Canvas Render Mode to World Space.
- 12. Create Background\_Image, ETA\_Text, PickupArea\_Text, and Status\_Text.
- 13. Use TextMeshProUGUI for all dynamic text fields.
- 14. Add BillboardReceiver to the billboard canvas.
- 15. Wire Time Text to ETA\_Text, Pickup Tag Text to PickupArea\_Text, and Status Text to Status\_Text.

5.5 Route endpoints

Add RouteEndpoint to semantic parking and pickup objects. The component links scene-level concepts to the real route-node network.

| Object         | Endpoint Id | Linked Route Node |
|----------------|-------------|-------------------|
| ParkingSpot_12 | P12         | Node_Parking_12   |
| Pickup_A       | A           | Node_Pickup_A     |
| Pickup_B       | B           | Node_Pickup_B     |

5.6 Pedestrian crossing metadata

Add RouteNodeMetadata only to route nodes that represent pedestrian crossings or interaction areas where the car is expected to slow down or wait.

| Field                           | Value | Purpose                                    |
|---------------------------------|-------|--------------------------------------------|
| Is Pedestrian Crossing          | true  | Marks the node as a crossing delay source. |
| Override Default Crossing Delay | false | Use CarETAOrchestrator default delay.      |
| Crossing Delay Seconds          | 8-15  | Used only when override is true.           |

6. ParkingTrafficProvider Setup

ParkingTrafficProvider estimates the traffic buffer using total cars and active cars. It supports two input modes: Manual Input for testing and External Traffic Orchestrator for production wiring.

| Mode                          | Usage                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------|
| Manual Input                  | Use Inspector values for total cars and active cars. Useful for standalone tests.       |
| External Traffic Orchestrator | Use values pushed by the main traffic orchestrator through SetExternalTrafficData(...). |

6.1 Delay model

```
occupancyRatio = activeCarsCount / totalCarsInSimulation
congestionFactor = 1 + occupancyRatio * congestionSensitivity
trafficDelay = fixedTrafficBaseDelaySeconds + activeCarsCount * delayPerActiveCarSeconds * congestionFactor
trafficDelay = clamp(trafficDelay, 0, maxTrafficDelaySeconds)
```

| Field                           | Recommended value                             | Purpose                         |
|---------------------------------|-----------------------------------------------|---------------------------------|
| Input Mode                      | Manual Input or External Traffic Orchestrator | Select the data source.         |
| Manual Total Cars In Simulation | 20                                            | Used only in Manual Input mode. |
| Manual Active Cars Count        | 5                                             | Used only in Manual Input mode. |

|                                  |    |                                                  |
|----------------------------------|----|--------------------------------------------------|
| Fixed Traffic Base Delay Seconds | 3  | Base delay always added when estimating traffic. |
| Delay Per Active Car Seconds     | 2  | Average delay contribution per active car.       |
| Congestion Sensitivity           | 1  | Amplifies delay as occupancy ratio increases.    |
| Max Traffic Delay Seconds        | 30 | Upper bound to avoid unrealistic ETA.            |

## 7. External Traffic Orchestrator Wiring

In production, do not manually edit the external traffic values in the Inspector. The main traffic orchestrator must hold a reference to ParkingTrafficProvider and push updated traffic data by code.

```
[SerializeField] private ParkingTrafficProvider parkingTrafficProvider;

// Called whenever traffic data is updated by the main orchestrator.
parkingTrafficProvider.SetExternalTrafficData(
    totalCarsInSimulation,
    activeCarsCount
);
```

16. Select the GameObject that contains the main traffic orchestrator.
17. Add a serialized reference field to ParkingTrafficProvider in that script.
18. Drag AHMI\_DynamicRouteSystem into the Parking Traffic Provider reference field.
19. On ParkingTrafficProvider, set Input Mode to External Traffic Orchestrator.
20. Call SetExternalTrafficData(totalCarsInSimulation, activeCarsCount) whenever traffic values change.

If the external fields are visible in the Inspector, treat them as runtime/debug state only. They are not production configuration fields. They may optionally be hidden in code with [HideInInspector] [SerializeField].

```
MainTrafficOrchestrator
-> SetExternalTrafficData(totalCarsInSimulation, activeCarsCount)
ParkingTrafficProvider
-> GetTrafficDelaySeconds()
CarETAOrchestrator
-> SimulationEvents
BillboardReceiver
```

## 8. Dynamic Route Integration

The external parking/pickup orchestrator must provide an ordered list of route nodes and the assigned pickup label.

```
dynamicRouteBridge.ReceiveRoute(generatedRouteNodes, pickupLabel);
```

Supported route input types:

- List<Transform>
- Transform[]
- List<GameObject>
- GameObject[]
- List<RouteEndpoint>
- RouteEndpoint[]

```
// Example
dynamicRouteBridge.ReceiveRoute(routeNodes, "A");
```

## 9. ETA Calculation Logic

```
finalETA = baseTravelTime
    + remainingPedestrianCrossingDelay
    + parkingTrafficDelay
    + safetyBuffer

baseTravelTime = remainingRouteDistance / speedMetersPerSecond
```

The orchestrator calculates internally at the configured internal interval, but publishes to the billboard only when needed.

- Publish immediately on first valid ETA.
- Publish if the billboard publish interval has elapsed.
- Publish if ETA changes by at least etaChangeThresholdSeconds.
- Publish if vehicle status changes.
- Publish immediately when the vehicle arrives.

The displayed ETA is rounded upward using Round ETA To Nearest Seconds to avoid a frustrating second-by-second countdown.

## 10. Standalone Debug Test

21. Add DynamicRouteDebugTester to AHMI\_DynamicRouteSystem.
22. Assign DynamicRouteBridge.
23. Fill Test Route Nodes with at least two real route nodes.
24. Set Test Pickup Label to A, B, or C.
25. Enter Play Mode and press R.

Expected billboard output:

```
Pickup A
~ETA value
Vehicle returning
```

## 11. Export Checklist

| Asset item                 | Export status    |
|----------------------------|------------------|
| SimulationEvents.cs        | Required         |
| CarETAOrchestrator.cs      | Required         |
| BillboardReceiver.cs       | Required         |
| DynamicRouteBridge.cs      | Required         |
| ParkingTrafficProvider.cs  | Required         |
| RouteEndpoint.cs           | Required         |
| RouteNodeMetadata.cs       | Required         |
| DynamicRouteDebugTester.cs | Optional debug   |
| TotemCarRequest.cs         | Optional trigger |

- Export AHMI\_DynamicRouteSystem as the main prefab/root object.
- Include all scripts listed above.
- Include billboard prefab only if the UI layout should be reused.
- Do not include project-specific route nodes unless exporting a full scene sample.
- After import into another project, rewire route nodes, billboard TextMeshPro references, and external orchestrator references.

## 12. Final Wiring Checklist

- DynamicRouteBridge -> CarETAOrchestrator assigned.
- CarETAOrchestrator -> ParkingTrafficProvider assigned.
- BillboardReceiver -> ETA\_Text, PickupArea\_Text, Status\_Text assigned.
- RouteEndpoint -> Linked Route Node assigned for parking and pickup objects.
- RouteNodeMetadata assigned to pedestrian crossing nodes.
- Main traffic orchestrator calls ParkingTrafficProvider.SetExternalTrafficData(...).
- Random route orchestrator calls DynamicRouteBridge.ReceiveRoute(...).
- No fixed WP\_\* folder or manual route array is required.